N32H78x 系列 MCU

车牌识别模型 (Yolo-FastestV2) 部署应用笔记

简介

此文档的目的在于让使用者能够快速熟悉在本项目（N32H78x 系列微控制器 + 外扩 SDRAM + OV5640 摄像头）上部署一套车牌检测识别模型的完整流程：从最初的 PyTorch 训练权重 .pth 开始，经过 ONNX 导出、INT8 全整型量化的 TFLite 转换，直到生成 C 数组 model_data.h 并嵌入 Keil 工程；同时说明工程本身的两种构建方式（纯 CPU 构建与部分使用 GPU 构建）以及上板后的串口输出与定期扫描节奏。

阅读本文件后，使用者应当能够独立完成一次“换模型 → 生成 model_data.h → 编译 → 烧录 → 用串口验证识别结果”的完整迭代。

【本版说明（V1.0.0，2026-10-22）】本文描述的是板端三网级联（① 整图找牌 → ② 裁剪找字 → ③ 逐字分类，共 561,584 参数）的完整流程、内存布局与实测口径，整条链路的规格以 mcu_code/RELEASE_v1.0_board.md 为准。第一级 ① 的转换工具链沿用单模型时期的路径：tflite/export_onnx.py 导 ONNX，tflite/onnx2tf_int8.py --onnx/--calib/--out-dir 量化，三个模型各一份。与更早的单模型固件文档相比，本版更正了这些内容：模型文件名与 MD5（第 1、2.4、2.5、3.1 节）、定期扫描周期（5 秒 → 一轮跑完歇 1 秒，周期约 2.7~3.5 秒）、开机行数与串口输出格式；并明确：本交付版不处理任何串口输入（DISPLAY_PLATE_ONLY = 1：开机只打两行，每轮只打车牌本身，按任何键都不会有反应）。

本文所有路径均以【工程根目录】为基准书写，不依赖工程被放在哪个盘符或目录下；把整个工程目录整体拷贝到任意位置后，文中命令与路径仍然成立。

目录

1. 概述
2. 模型转换工具链
    2.1 总体流程
    2.2 数据集与训练（training/ 下的 data/、genanchors.py、train.py）
    2.3 .pth → ONNX（export_onnx=True 的 fused 导出 + onnxsim）
    2.4 ONNX → INT8 TFLite（onnx2tf，真实车牌图校准）
    2.5 TFLite → C 数组 model_data.h（export_c_array.py）
3. 工程集成
    3.1 工程目录结构与路径约定
    3.2 文件与目录
    3.3 内存布局（M7_TCM.sct、tensor_arena、data_area）
    3.4 推理、解码与车牌拼接（yolo_inference.cpp）
    3.5 编译期开关（GPU_ENABLE、BOOT_VERBOSE）
4. 工程使用
    4.1 纯 CPU 构建（GPU_ENABLE = 0）
    4.2 部分使用 GPU 构建（GPU_ENABLE = 1）
    4.3 开机输出与定期扫描
    4.4 串口输入（本交付版无按键）
    4.5 典型使用流程
5. 常见问题
版本历史

1. 概述

本文介绍把一套 Yolo-FastestV2 车牌检测模型（shufflenetv2 主干 + 两个检测头）从训练权重一路部署到国民技术 N32H78x 应用工程的方法，包含工具链的每一个环节与工程侧的使用方式。

1.1 硬件与软件构成

项目	内容
目标板	N32H78x (Cortex-M7，主频 600 MHz) + 外扩 SDRAM
摄像头	OV5640 (DVP 接口)，输出 RGB565 帧
推理框架	TFLite Micro 1.26.x（Keil RTE / cortex_m_generic pack）
模型输入	320 × 320 × 3，RGB888，int8
模型输出	两个检测头 (1,20,20,85) stride16 + (1,10,10,85) stride32
类别数	70 类车牌字符（0 = 车牌整体框；1~34 数字与字母；35~69 省份汉字）
模型文件	tflite/onnx2tf_int8_plate/plate-int8.tflite，431,360 字节，MD5 82693BCE77C3D5A9099BF268B5D815F5（V1.0.0 镜像里的 ①）。注意：tflite/yolo-plate-int8.tflite 是另一份权重，字节数相同但内容不同（MD5 5193E19A…），不在本镜像里
IDE / 编译器	Keil MDK（armclang），工程文件 mcu_code/Printf2/MDK-ARM/Printf.uvprojx

1.2 三个阶段

整个方案可以分成三个阶段，本文按这三个阶段组织：

① 训练阶段——在 PC 上用 PyTorch 训练 Yolo-FastestV2，得到权重 .pth（第 2.2 节）。

② 转换阶段——.pth 先导出 ONNX；ONNX 经 onnx2tf 做全整型量化（用真实车牌校准图）得到 INT8 TFLite，再由 export_c_array.py 生成 C 数组 model_data.h（第 2.3 ~ 2.5 节）。

③ 部署阶段——把 model_data.h 与推理代码嵌入 Keil 工程，编译、烧录，用串口命令验证（第 3、4 章）。

2. 模型转换工具链

2.1 总体流程

与本方案相关的完整链路如下（各路径均相对于工程根目录）。训练侧的 data/、model/、utils/、img/、weights/ 与各训练脚本都收在 training/ 子目录下，其余环节在工程根目录的 pytorch2onnx/ 与 tflite/ 下：

```
     training/data/voc.data  (70 类, 320x320)  +  training/data/voc.names
                                |
                          train.py
                                |
                                v
training/weights/voc-299-epoch-0.270855ap-model.pth     (.pth)
                                |
               tflite/export_onnx.py --kind detector
               Detector(classes, anchor_num, load_param, export_onnx=True)
                                |
                                v
                 tflite/onnx/plate.onnx
               (输入 x，两个 85 通道输出，sigmoid/softmax 已烘进图里；
                单文件，没有 .onnx.data 附属文件)
                                |
                                v
     tflite/onnx2tf_int8.py --onnx ... --calib ... --out-dir ...
                 (onnx2tf, full-int8, 整图校准)
                                |
                                v
                  tflite/onnx2tf_int8_plate/
                    plate-int8.tflite
                         (431,360 B, MD5 82693BCE…)
                                |
                                |
                tflite/export_c_array.py
                                |
                                v
           mcu_code/Printf2/inc/model_data.h
                                |
                                v
   Keil 工程 (mcu_code/Printf2/MDK-ARM/Printf.uvprojx)
                                |
                           编译 (armclang)
                                |
                                v
      Printf.axf / Printf.bin  --->  使用者用 uVision 烧录到板子
```

说明：

这条路线就是进入固件的那一条，产物是 model_data.h。V1.0.0 的 ① 用 tflite/export_onnx.py 导出 tflite/onnx/plate.onnx（输入名同样是 x、两个 85 通道输出，见 2.3.4；旧路线见 2.3）；**上板那一个模型文件是 tflite/onnx2tf_int8.py 产出的**（onnx2tf + 300 张真实车牌图校准），输出目录 tflite/onnx2tf_int8_plate/，见 2.4。

第 2 章分两处执行。**训练侧**（genanchors.py、train.py、test.py、evaluation.py、test_onnx.py、test_onnx_opt.py 以及它们读写的 data/、model/、utils/、img/、weights/）都在 training/ 子目录下，它们读写的 data/、model/ 这类相对路径都以 training/ 为基准解析。**pytorch2onnx/ 与 tflite/ 下的脚本**按脚本自身所在位置定位路径。两类脚本在开头都会把工作目录切到脚本自己所在的目录，因此在任意工作目录下执行都可以。

2.2 数据集与训练

2.2.1 数据集与配置文件

数据集格式与 Darknet YOLO 一致：每张图片配一个同名 .txt，内容为“类别 cx cy w h”（归一化）。本项目的配置文件为 training/data/voc.data：

```
[training/data/voc.data]
  [model-configure]
  classes=70                  # 类别数 = 70
  width=320                   # 模型输入宽
  height=320                  # 模型输入高
  anchor_num=3                # 每个检测头 3 个 anchor
  anchors=11.03,24.54, 14.40,36.74, 20.08,52.82,     # stride16 三个
          27.04,78.23, 144.02,96.56, 239.25,195.42   # stride32 三个
  [data-configure]
  train=data/train.txt
  val=data/val.txt
  names=data/voc.names
```

文件里 train / val / names 三条路径，以及 train.txt / val.txt 里每一行的图片路径，都相对 training/ 解析（脚本启动时会自己把工作目录切到 training/，所以在哪个目录下执行都一样）——全部是相对路径，整个 training/ 搬到任何位置都不用改这些文件。training/data/voc.names 共 70 行，顺序必须与模型输出通道一一对应：第 1 行 plate（车牌整体框），第 2~11 行为数字 0~9，第 12~35 行为字母 A~Z（去掉 I 与 O），第 36~70 行为省份汉字。这份顺序同时被固件里的 CHARS[] 与 PROV_GBK[35][2] 两张表使用（见 3.4 节）。

2.2.2 训练与评估

环境：conda 环境 Yolo-FastestV2，执行 pip install -r requirements.txt 安装依赖（requirements.txt 也在 training/ 下）。常用命令如下 —— 脚本开头会自己把工作目录切到 training/，所以在任意目录下执行都一样；命令里写的是相对 training/ 的路径：

```
# 1) 根据当前数据集生成 anchor
python genanchors.py --traintxt ./data/train.txt
#    产物 ./anchors6.txt ：第一行是 6 个 anchor，第二行是平均 IoU

# 2) 训练
python train.py --data data/voc.data
#    产物 weights/voc-<epoch>-epoch-<ap>ap-model.pth

# 3) 单图测试 / mAP 评估
python test.py       --data data/voc.data --weights weights/voc-299-epoch-0.270855ap-model.pth
--img img/21.jpg
python evaluation.py --data data/voc.data --weights weights/voc-299-epoch-0.270855ap-model.pth
```

本项目 V1.0.0 实际采用 training/weights/voc-299-epoch-0.270855ap-model.pth（① 整图找牌），裁剪找字用 weights/voc_crop_ft-119-epoch-0.589619ap-model.pth，逐字分类用 weights/charhead_16_32_64_32_64.pt。文件名里的 0.271 是 train.py 写进文件名的验证集 AP，不是最终部署指标：V1.0.0 冻结口径是整串 78.99%、7 字桶 83.7%（training/RELEASE_v1.0_2stage.md）。上一代 voc-290-epoch-0.273037ap-model.pth 仍留在 weights/ 作对照（2026-10-14 用当前 data/val.txt 与当前评估代码重跑得 70 类 mAP 0.199，与文件名不符，原因是验证集清单/评估代码漂移，不是权重本身）；换成 voc-299 后整串高约 2 点。train.py 的 --data 既决定类别数与输入尺寸，也决定 anchor，改动后必须重跑导出流程。

2026-10-14 训练配方改了两处，**两处都不在上面那份出厂权重里**（那份是 2026-08-29 训的），只在重新训练时生效，且都不改变导出的 ONNX / TFLite 结构：

training/utils/datasets.py 的对比度/亮度增强修了一个真 bug：beta 原来写成 random.uniform(0.25, 1.75)，而 cv2.addWeighted 的 beta 是**灰度级（0~255）不是倍率**，所以亮度抖动实际不到 2/255、等于没做，现已改为 ±32；同时 img_aug 启用 augment_hsv，并保持 motion_blur 与 random_resize 是注释掉的 —— 前者的模糊核会把 320 输入里中位只有 13 px 的字符整个糊掉，后者是先缩到 0.8 再放回来，两者都会让模型学到训练集里不存在的退化。

training/utils/loss.py 的分类交叉熵改为按类别频率加权（权重由 utils.utils.class_weights() 从 train.txt 的标注统计出来）：w = clamp((字符类中位频次 / n_c) 的 0.5 次方, 1.0, 4.0)，**只上抬稀有类** —— 常见字符类和 plate 类（n=4461）权重恒为 1.0，分类损失的整体尺度不变；下界刻意取 1.0 而不是按频率均衡，因为 plate 类现在是唯一可用的输出（AP 0.68），不能被压掉。动机是出厂权重 70 类里有 18 类在验证集上 AP 恰好为 0.00000（16 个稀有省份加字母 W、T），模型根本不发这些字符，而它们在训练集里只有 5~160 个框，8、9 两类却各约 2000 个。权重为 1.0 时该改动与改动前逐位相同。

2.3 .pth → ONNX（export_onnx=True 的 fused 导出）

这一步决定了整条链的输入形态，先看清 Detector 的两个输出形态。

2.3.1 Detector 的 forward 形态（关键）

training/model/detector.py 的构造函数签名是 Detector(classes, anchor_num, load_param, export_onnx=False)，上板用的这一支把第 4 个参数 export_onnx 传 True：

export_onnx	forward 返回	用途
True（上板用）	两个已融后处理的张量 cat(reg|obj|cls).permute(0,2,3,1)，形状 [1,20,20,85] 与 [1,10,10,85]，sigmoid(reg)、sigmoid(obj)、softmax(cls) 全部烘进图里，输入名 x	上板量化用的就是这一支

工程根目录 pytorch2onnx/ 下的 pytorch2onnx.py 与 pytorch2onnx_simplify.py 都按这一写法显式传了第 4 个参数（export_onnx=True），可直接使用；自己另写脚本时照此即可。

2.3.2 导出上板用的 fused ONNX

旧路线（只导 ① 的手工对照）可用工程自带的两个脚本（都在 pytorch2onnx/ 下；它们按脚本自身位置定位路径，命令在哪个目录下执行都可以）；V1.0.0 的正式导出是 tflite/export_onnx.py（见 2.3.4）：

```
# (a) 只导出，不做图简化
python pytorch2onnx/pytorch2onnx.py
#   -> pytorch2onnx/yolo-fastestv2.onnx
#      裸导出时权重会拆到同名的 yolo-fastestv2.onnx.data 附属文件里（本模型约 945976 字节），
#      搬动这个模型时两个文件要一起拷；经 onnxsim 之后才会内联成单文件

# (b) 导出 + onnxsim 一步到位（旧路线用这一条；V1.0.0 见 2.3.4）
python pytorch2onnx/pytorch2onnx_simplify.py --data ./training/data/voc.data \
    --weights ./training/weights/voc-299-epoch-0.270855ap-model.pth \
    --output  ./pytorch2onnx/yolo-fastestv2.onnx
#   这两个参数已是脚本里的默认值，不带参数直接跑也一样
```

```
#   -> 先得到 pytorch2onnx/yolo-fastestv2.onnx，再自动简化成
#      pytorch2onnx/yolo-fastestv2-opt.onnx（单文件，权重已内联）
```

两个脚本内部用的 torch.onnx.export 等价写法如下，自己写脚本时照此即可：

```
import sys, torch
sys.path.insert(0, "training")   # model/ utils/ 两个包在 training/ 下
from model.detector import Detector
from utils.utils import load_datafile

cfg = load_datafile("training/data/voc.data")
net = Detector(cfg["classes"], cfg["anchor_num"], True, True)   # 第 4 个参数必须为 True
net.load_state_dict(torch.load("training/weights/voc-299-epoch-0.270855ap-model.pth",
map_location="cpu"))
net.eval()

torch.onnx.export(
    net,
    torch.randn(1, 3, 320, 320),          # NCHW [1,3,320,320] float32
    "pytorch2onnx/yolo-fastestv2.onnx",
    export_params=True,
    opset_version=18,
    do_constant_folding=True,
    # 不传 input_names / output_names：输入名取 forward 的参数名 x，
    # 两个输出自动命名为 permute_13 / permute_14。下游 onnx2tf 的校准数据是按名字 "x" 绑定的（见
2.4）
)

# 图结构简化（旧路线：V1.0.0 用 tflite/export_onnx.py，不需要这一步）
python -m onnxsim pytorch2onnx/yolo-fastestv2.onnx pytorch2onnx/yolo-fastestv2-opt.onnx
```

2.3.3 导出后核对的指纹

用 Netron 打开，或 print(onnx.load(path).graph) 核对下面这四项，与 2.3.1 的要求一致：

检查项	正确值
输入	x [1, 3, 320, 320]
输出	permute_13 [1,20,20,85]、permute_14 [1,10,10,85] —— 两个 85 通道的 NHWC 张量
图中算子	Sigmoid × 4、Softmax × 2（对应 reg2/obj2/reg3/obj3 与 cls2/cls3）
节点数	约 219

V1.0.0 的 ① 用的 ONNX 是 tflite/onnx/plate.onnx（994,634 字节，MD5 8575D3042683DBF4AA0A77B4873E7297），由 tflite/export_onnx.py 导出；量化脚本直接读这一份，下游产物就是上板的 tflite/onnx2tf_int8_plate/plate-int8.tflite。pytorch2onnx/yolo-fastestv2-opt.onnx 是旧路线（pytorch2onnx_simplify.py + onnxsim）的产物，当前文件 994,653 字节、MD5 9F082E29DE1D69B9A59015129FD4327A，与 plate.onnx 字节不同、但图与权重完全相同（2026-10-21 实测：161 个 initializer 逐位一致，19 字节差只在 ONNX 的 producer 元数据；重跑 export_onnx.py 得到的字节就等于 opt.onnx，清掉 producer 字段才是 plate.onnx 的 8575D304…），量化只读图，两者结果一致。早期本文写的 MD5 845EC6BD… 属于更早一次导出，已经对不上。

关于附属文件：裸导出的 pytorch2onnx/yolo-fastestv2.onnx 只有约 458561 字节，权重被拆在同名的 yolo-fastestv2.onnx.data（本模型 945976 字节）里，**搬运模型时必须两个一起拷**；经 onnxsim 简化后权重会内联回单文件 —— pytorch2onnx/yolo-fastestv2-opt.onnx 就是单文件。

本工程用 training/ 下的 test_onnx.py 与 test_onnx_opt.py 验证这一步的产物（这两个脚本会自己把工作目录切到 training/，在任意目录下执行都一样）：前者把 PyTorch 与 ONNX 在同一张图上给出的两个 85 通道输出逐元素对比；后者不依赖 PyTorch，直接解 ONNX 输出的框，并会先检查输入名是否为 x、输出是否为两个 85 通道张量。两者用的是不同的后处理（test_onnx.py 用工程自带的 non_max_suppression，对 class 做偏移后批处理 NMS；test_onnx_opt.py 自带一套贪心 NMS），所以同一张图上解出的框数不一定完全相同，看误差数值和框的位置即可。

导出的输入尺寸取自 training/data/voc.data 的 width/height（本项目 320×320）：anchor 与 stride 都按这个尺寸定义，改动 training/data/voc.data 之后需要重新导出。

2.4 ONNX → INT8 TFLite

这是上板路线的核心一步：底层调用 onnx2tf 把 fused ONNX 转成 TFLite，再做 full-integer（全整型）量化。**上板的那个 tflite/onnx2tf_int8_plate/plate-int8.tflite 是 tflite/onnx2tf_int8.py 产出的。**

输入是 2.3 导出的 fused ONNX（输入名 x、两个 85 通道输出、sigmoid/softmax 已内嵌），形态见 2.3.1。

2.4.1 onnx2tf_int8.py（上板模型的产出脚本）

位置：tflite/onnx2tf_int8.py（工程根目录下的 tflite/ 目录）。V1.0.0 起它是参数化脚本：--onnx、--calib、--out-dir 都是必填（--size 只在 --calib 给目录时用得上），三个模型各给各的。它做的是“用真实图片做校准的全整型量化”，关键点有三条：

校准集用真实图片：从 training/data/val/*.jpg 取图（不足 300 张再补 training/data/train），PIL 双线性缩放到 320×320、/255 归一化，random.seed(7) 固定抽样顺序，把 300 张 × 320×320×3 的 float32 存成 training/model/calib_real_300.npy 并同时喂给转换器 —— 真实车牌图才能给出可信的整型尺度。

校准数据按输入张量名绑定：custom_input_op_name_np_data_path 里第一项就是 "x"（配 calib_real_300.npy，取值区间 0..1），所以上游 ONNX 的输入名是 x —— 与 2.3.2 里不传 input_names 的导出写法一致。

input_quant_dtype 与 output_quant_dtype 都是 int8，not_use_onnxsim=True，tflite_backend="tf_converter"；输出目录 tflite/onnx2tf_int8_plate/（V1.0.0 起 --out-dir 是命令行参数，① 用的就是这一目录）。

2.4.2 使用方法

```
# 在【工程根目录】下执行（脚本按自身所在位置定位路径，在任意目录下执行都可以）
python tflite/onnx2tf_int8.py --onnx tflite/onnx/plate.onnx \
    --calib training/data/val --size 320x320 --out-dir tflite/onnx2tf_int8_plate
#   读 : tflite/onnx/plate.onnx                          （2.3 的导出产物，直接读）
#   写 : tflite/onnx2tf_int8_plate/_calib.npy            （校准缓存；不带 --keep-npy 转换后删除）
#        tflite/onnx2tf_int8_plate/                          （7 个 tflite + saved_model）
#   上板：tflite/onnx2tf_int8_plate/plate-int8.tflite
#         （--out-dir 已直接写在这一目录，无需再拷贝）
```

路径对应关系：脚本按 --onnx 直接读 2.3 的导出产物（V1.0.0 是 tflite/onnx/plate.onnx），按 --calib 取校准集，按 --out-dir 写产物，中间不需要拷贝（见 2.1 的流程图）。第一次跑会重新生成

校准集，耗时以分钟计。②③ 换成 tflite/onnx/crop.onnx、tflite/onnx/charhead.onnx 以及各自的校准集（② 用裁剪块，③ 用 32x64 字符块）即可。

产物目录 tflite/onnx2tf_int8_plate/ 里放着同一张图的多种精度版本，上板的是 plate-int8.tflite（即 full_integer_quant 那一份）：

属性	值
文件	tflite/onnx2tf_int8_plate/plate-int8.tflite
大小 / MD5	431,360 字节 / 82693BCE77C3D5A9099BF268B5D815F5
来源	tflite/onnx2tf_int8_plate/plate-int8.tflite，full-integer 量化产物（onnx2tf_int8.py 的 --out-dir 直接产出，无需改名）
输入	int8 [1,320,320,3] NHWC，quant scale = 1/255，zero point = -128
输入像素填法	dst[i] = (int8_t)(rgb_pixel - 128)
输出	两个 int8 head，布局 [reg12 | obj3 | cls70]；sigmoid/softmax 已内嵌
输出反量化	float = (q + 128) / 255

2.5 TFLite → C 数组 model_data.h

脚本：tflite/export_c_array.py。功能：把 .tflite 二进制逐字节读入，生成 C 头文件，内容是每行 16 字节的数组初始化列表、模型长度常量，以及模型摘要注释。

功能要点

```
// Auto-generated from plate-int8.tflite
// bytes: 431360   MD5: 82693BCE77C3D5A9099BF268B5D815F5

// yolo-plate int8 model. Placed in XIP Flash region 'data_area'
__attribute__((section("data_area"), aligned(16)))
const unsigned char g_model_data[] = { 0x1c, 0x00, 0x00, 0x00, 0x54, 0x46, 0x4c, 0x33, ... };

const unsigned int g_model_data_size = 431360;
```

数组带 section("data_area") 属性 —— 模型落在分散加载文件定义的 XIP Flash 段里。

数组带 aligned(16) —— TFLite Micro 按 16 字节对齐读取模型。

脚本自动计算并写入 MD5，便于与 tflite/onnx2tf_int8_plate/plate-int8.tflite 逐字节对账。

使用方法

```
# 在【工程根目录】下执行（脚本按自身所在位置定位路径，在任意目录下执行都可以）
python tflite/export_c_array.py --tflite tflite/onnx2tf_int8_plate/plate-int8.tflite \
    --array g_model_data --section data_area --out mcu_code/Printf2/inc/model_data.h
#  读入 : tflite/onnx2tf_int8_plate/plate-int8.tflite
#  产出 : mcu_code/Printf2/inc/model_data.h   (直接写入固件头文件目录)

# 也可显式指定
python tflite/export_c_array.py <tflite 路径> <输出目录>
```

该脚本直接写入 mcu_code/Printf2/inc/model_data.h，也就是 Keil 工程真正 #include 的那一份，换模型后不需要再手工拷贝文件。

注意：export_c_array.py 不带 --tflite 时默认读的就是上面的 ①（tflite/onnx2tf_int8_plate/plate-int8.tflite）并写 inc/model_data.h；换 ②③ 必须显式给 --tflite、--array 与 --out，否则会把 ① 的数组悄悄覆盖掉。（2026-10-20 之前这个默认值指向 tflite/yolo-plate-int8.tflite —— 那份 2026-09 的旧权重与 ① 同为 431,360 字节但内容不同，照默认跑一次就会把 ① 换掉。已修正。）

V1.0.0 固件里有三份模型数组：inc/model_data.h（g_model_data，①，段 data_area）、inc/model_data_crop.h（②）与 inc/model_data_charhead.h（③），后两份在段 model_ro。换 ① 时只覆盖 model_data.h，不要动另外两份；三个数组名互不相同，不会撞名。

3. 工程集成

3.1 工程目录结构与路径约定

本文所有路径都相对于【工程根目录】。工程可以整体拷贝到任意盘符或目录下，只要保持下面的相对结构，文中所有命令与路径都继续成立。

```
<工程根目录>/
├── training/                          训练环节（2.2 节；脚本自己切到 training/，任意目录下可执行）
│   ├── data/                          数据集与配置
│   │   ├── voc.data                   类别数 / 输入尺寸 / anchor / 数据列表
│   │   ├── voc.names                  70 类字符名（顺序即类别索引）
│   │   ├── train/  val/               图片与同名 .txt 标注
│   │   └── train.txt / val.txt        训练/验证图片路径列表
│   ├── weights/                       训练产出的 .pth 权重
│   ├── model/                         模型定义与转换产物
│   │   ├── calib_real_300.npy         量化校准集（2.4 首次运行时生成）
│   │   ├── voc-290 …-model.pth        旧权重（weights/ 下有；V1.0.0 的 ① 用 voc-299…）
│   │   └── backbone/  detector.py  fpn.py 网络定义（backbone/ 内还有 backbone.pth）
│   ├── utils/                         数据集与解码工具（各转换脚本 import 依赖）
│   ├── img/                           测试图片
│   ├── genanchors.py  train.py  test.py  evaluation.py
│   └── test_onnx.py  test_onnx_opt.py 验证 2.3 的 ONNX 产物
├── pytorch2onnx/                      导出脚本 + ONNX 产物
│   ├── pytorch2onnx.py                .pth → ONNX（只导出）
│   ├── pytorch2onnx_simplify.py       .pth → ONNX + onnxsim
│   ├── yolo-fastestv2.onnx(.data)     2.3 的裸导出产物（权重在 .data 里）
│   └── yolo-fastestv2-opt.onnx        旧路线的简化产物（V1.0.0 量化读 tflite/onnx/plate.onnx）
├── tflite/                            TFLite 环节（量化 + 上板模型 + C 数组生成）
│   ├── onnx2tf_int8.py                ONNX → INT8 TFLite（2.4）
│   ├── onnx2tf_int8_plate/            ★ 量化产物目录（7 个 tflite + saved_model）
│   │   └── plate-int8.tflite          → 上板的那一份（①）
│   ├── onnx2tf_int8_crop/ onnx2tf_int8_charhead/  ②③ 的量化产物目录
│   └── export_c_array.py              TFLite → model_data.h（2.5）
└── mcu_code/
    ├── tools/                         离线分析脚本（只读 tflite；内部排查用，交付包不含）
    └── Printf2/                       ★ 固件工程
        ├── inc/                       头文件（含 model_data.h、gpu.h、Message.h）
        ├── src/                       源文件（main.c、camera.c、gpu.c、bsp_*.c）
        └── MDK-ARM/                   Keil 工程（Printf.uvprojx、yolo_inference.cpp、
                                       M7_TCM.sct、yolo_inference.h）
```

（V1.0.0 另有 ②③ 两个模型：tflite/onnx2tf_int8_crop/crop-int8.tflite 与 tflite/onnx2tf_int8_charhead/charhead-int8.tflite，产物目录同名，C 数组分别是 inc/model_data_crop.h 与 inc/model_data_charhead.h，段落为 model_ro；完整规格见 mcu_code/RELEASE_v1.0_board.md。）

路径约定：

训练侧脚本（2.2 节：genanchors.py、train.py、test.py、evaluation.py、test_onnx.py、test_onnx_opt.py）读写的 data/、model/、utils/、img/、weights/ 都在 training/ 里；脚本开头会自己把工作目录切到脚本所在的 training/，所以在任意工作目录下执行都可以。

转换脚本（2.3~2.5 节：pytorch2onnx/*.py、tflite/*.py）按脚本自身位置定位路径，在任意工作目录下执行都可以；命令中给出的路径相对工程根目录。

固件侧路径（第 3、4 章）相对 mcu_code/Printf2/；Keil 工程自身用的相对路径则由 .uvprojx 内部定义，不要手工改动。

3.2 文件与目录

固件工程 mcu_code/Printf2 中与模型部署相关的文件如下：

文件（相对 mcu_code/Printf2）	作用
inc/model_data.h	车牌 int8 模型的 C 数组与长度常量（模型本体）
MDK-ARM/yolo_inference.cpp	TFLite Micro 推理 + 解码 + NMS + 车牌字符串拼接
inc/yolo_inference.h	对外接口：yolo_plate_init / yolo_plate_detect / yolo_plate_read 等
MDK-ARM/M7_TCM.sct	分散加载文件，定义 tensor_arena 与 data_area 的落址
inc/gpu.h	GPU 总开关 GPU_ENABLE 与各运行时开关的声明
src/gpu.c	2.5D GPU 驱动封装与各算子卸载实现
src/camera.c / inc/camera.h	摄像头抓帧（CPU 双线性缩放 / GPU 缩放 blit 两条前端）
src/main.c	上电初始化、主循环、串口命令分发
src/n32h7xx_it.c	中断（DVP、GPU）与看门狗
inc/model_data_crop.h	② 裁剪找字模型的 C 数组（段 model_ro）
inc/model_data_charhead.h	③ 逐字分类模型的 C 数组（段 model_ro）

Keil 集成步骤：

添加源文件 MDK-ARM/yolo_inference.cpp，添加头文件目录 ..\inc。

确认 M7_TCM.sct 中 tensor arena 落在 SDRAM 的 UNINIT 区。

确认 yolo_inference.cpp 的算子解析器已注册全部所需算子（Conv2D / DepthwiseConv2D / MaxPool2D / Pad / Concatenation / Transpose / Reshape / Gather / ResizeNearestNeighbor / Logistic / Softmax / Quantize / Dequantize）。

3.3 内存布局

模型本体（431 KB）放在 XIP Flash 的 data_area 段，不占 RAM；张量竞技场（tensor arena）放在 SDRAM。yolo_inference.cpp 中的定义为（下面是 ① 的；V1.0.0 的 ② 另有一块 2 MB arena，③ 用 512 KB 区，两级的权重都在 model_ro 段、由 yolo_cascade_init() 拷进 SDRAM，完整布局见 RELEASE_v1.0_board.md 第 1 节）：

```
#ifndef TENSOR_ARENA_SIZE
#define TENSOR_ARENA_SIZE (0x200000)          // 2 MB
#endif

static uint8_t tensor_arena[TENSOR_ARENA_SIZE]
        __attribute__((section("tensor_arena_area"), aligned(16)));
```

SDRAM 必须在 yolo_plate_init() 之前由 BSP 完成 FEMC 初始化。arena 必须不小于模型峰值中间张量；若板载 SDRAM 容量不足，需同时修改 yolo_inference.cpp 的 TENSOR_ARENA_SIZE 与 M7_TCM.sct 中对应段的长度。0xC0000000 起的内存区域按 TFLM 的要求配置 MPU 属性（非对齐访问）。

3.4 推理、解码与车牌拼接

yolo_inference.cpp 对外提供三个接口：

```
#include "yolo_inference.h"

uint8_t rgb[320 * 320 * 3];                 // 320x320 RGB888 (0~255)

if (!yolo_plate_init()) { /* 初始化失败 */ } // 上电一次

while (1) {
    /* ... 抓图并缩放到 rgb ... */
    YoloBox boxes[64];
    int n = yolo_plate_detect(rgb, boxes, 64);     // 推理 + 解码 + NMS，返回框数
    char plate[YOLO_PLATE_MAXLEN + 1];
    int len = yolo_plate_read(boxes, n, plate, sizeof(plate));  // 拼成 GBK 车牌串
}
```

yolo_plate_init()：tflite::GetModel(g_model_data) → MicroInterpreter 构造 → AllocateTensors。失败时打印 init FAIL / yolo init ERROR，绝不静默。

yolo_plate_detect()：把 rgb 减 128 填成 int8 输入，invoke，再对两个输出头做解码与 NMS；坐标是 320×320 输入图上的像素。

yolo_plate_read()：跳过 class_id == 0 的整车牌框，对字符框按中心 x 排序后拼接。字符映射用两张静态表：

```
static const char CHARS[] = "0123456789ABCDEFGHJKLMNPQRSTUVWXYZ";  // class 1..34
static const uint8_t PROV_GBK[35][2] = { ... };                    // class 35..69（GBK 双字节）
```

串口终端需按 GBK 解码才能正确显示省份汉字。

3.5 编译期开关

宏	位置（相对 mcu_code/Printf2）	取值与含义
GPU_ENABLE	inc/gpu.h:57	1 = 交付产品构建（GPU 参与前端缩放 blit 与滤波；算子卸载默认仍关闭）；0 = 纯 CPU 构建，gpu.c 整个不参与链接
BOOT_VERBOSE	inc/Message.h:52	0（默认）= 开机只打 5 行；1 = 打开全部开机诊断打印。失败行与三个阶段标记不受此宏控制
CAMERA_ENABLE	inc/camera.h	1 = 走摄像头抓帧与定期扫描路径
AUTO_SCAN_PERIOD_US	src/main.c:704	取值 1 秒（= 1 * 1000 * 1000 微秒，V1.0.0 交付值）= 每轮跑完后的空闲等待，从上一轮结束算起（不是扫描间隔），所以实际周期 = 一轮耗时 + 本值。本交付版没有 “i”（同帧重跑）键，每轮都重新抓帧；① 是否沿用上一轮的牌框由 yolo_inference.cpp 的 YOLO_PLATE_REUSE_AGE = 5 加内容闸自动决定 —— ① 重跑的一轮约 2.48 秒（周期约 3.5 秒），沿用旧框的一轮约 1.71 秒（周期约

		2.7 秒）。开机第一轮立刻开始，不等一个周期。只改等待、不动推理，因此不改变任何识别结果；5 秒 → 1 秒是 2026-10-20 的实测调整，再往下调收益有限（推理本身已是下界）。
GPU_FILTER_ENABLE	inc/gpu.h	1 = 前端在缩放 blit 之后加一道可分离卷积低通滤波
DISPLAY_PLATE_ONLY	inc/Message.h:93	1（V1.0.0 交付值）= 串口只出车牌本身，开机两行；0 = 恢复全部诊断打印（含每轮 #序号 / n / grab / infer）。串口本来就不处理按键（见 4.4）。排查前先改成 0

4. 工程使用

工程支持两种构建：纯 CPU 构建与部分使用 GPU 构建。两者的差别只有一个编译宏 GPU_ENABLE，但产生的镜像、开机日志与可用开关都不同。

4.1 纯 CPU 构建（GPU_ENABLE = 0）

修改 mcu_code/Printf2/inc/gpu.h 第 57 行为 #define GPU_ENABLE 0，重新编译即可。特点：

产品路径完全不触碰 GPU：不做 GPU 初始化与自检，不注册任何 GPU 算子，抓帧回退到 CPU 双线性缩放。

镜像中 gpu.o 被链接器整体丢弃（判据：map 中 gpu.o 的组件尺寸行全 0，且没有任何产品目标文件引用它）。

开机打印 firmware: CPU，随后 boot ok。

串口输入：本交付版不处理任何串口输入 —— 收到的字节直接读掉丢弃，按任何键都不会有反应，也不会改变任何开关（见 4.4）。

4.2 部分使用 GPU 构建（GPU_ENABLE = 1）

这是交付产品的构建。GPU（NSGPU / D-AVE HD 2D）在上电时完成初始化与往返自检，并承担两件事：

抓帧前端：把 800×480 的相机帧缩放 blit 到 320×320（GPU 侧比例 103/256 = 2.4922），并可再走一道低通滤波（GPU_FILTER_ENABLE）。

算子卸载：深度卷积（g）、点卷积（c）、TRANSPOSE（j）等可以逐类搬到 GPU 上运行，但默认全部关闭。本交付版没有运行时开关，交付镜像里这些卸载始终保持关闭（见 4.4）。

开机打印 firmware: CPU+GPU，随后 boot ok。卸载开关的复位值就是交付镜像的实际状态：全部关闭（见 4.4）。

为什么卸载默认关闭：GPU 纹理字段只有 5 位无符号，每读一次激活都会截掉低 3 位，而它跑在串联的多层上，误差会逐层放大 —— 逐层自检合格并不等于整链合格。当前 2.5D GPU 的定位是“参与前端图像处理 + 提供可验证的算子卸载能力”，而不是默认承担全部卷积运算。

4.3 开机输出与定期扫描

BOOT_VERBOSE = 0（默认）时，GPU 构建开机共 5 行（V1.0.0 交付版另有 DISPLAY_PLATE_ONLY = 1，把其中 3 行也拦掉了，实际只看到两行，见本节末尾）：

```
yolo start ...
yolo init OK
gpu init OK
firmware: CPU+GPU
boot ok
```

纯 CPU 构建为 4 行（无 gpu init OK）。失败行（init FAIL / yolo init ERROR / camera init ERROR / gpu init ERROR <n> / gpu selftest FAIL <n> 等）在默认配置下都会打印；V1.0.0 交付版 DISPLAY_PLATE_ONLY = 1 时只有 yolo init ERROR 仍被放行，camera init ERROR 也会静默（见本节末尾）。

上电后主循环会自行定期扫描：一轮跑完歇 1 秒（AUTO_SCAN_PERIOD_US，从上一轮结束算起，不是扫描间隔），再抓一帧、跑一次推理，并打印一行结果，无人值守时日志不会为空。本交付版没有 “i”（同帧重跑）键，每轮都重新抓帧；① 是否沿用上一轮的牌框由 yolo_inference.cpp 的 YOLO_PLATE_REUSE_AGE = 5 加内容闸自动决定。板端实测一轮 1.71 秒（① 沿用旧框）/ 2.48 秒（① 重跑），周期约 2.7 / 3.5 秒：

```
#1 cas plate: 粤R888G8    n 7 grab 325000 us infer 2150000 us
#2 cas plate: (no char)   n 0 grab 323000 us infer 2130000 us
```

每行含义：#序号 / cas 或 one（这一轮走三网级联，还是级联没用上、退回单模型）/ 车牌字符串（无字符时打 (no char)）/ n（级联时是识别出的字符数，单模型时是框数）/ 抓帧耗时 / 推理耗时（微秒）。上面两行是 DISPLAY_PLATE_ONLY = 0 时的格式，数值随板而异；交付版默认 DISPLAY_PLATE_ONLY = 1，这一行不打。

这一行本身也是心跳：只要板子活着就会持续出现。

复制日志请用串口终端菜单里的“保存到文件”或鼠标拖选。

【V1.0.0 交付版的差异】inc/Message.h 的 DISPLAY_PLATE_ONLY = 1 时：开机只剩 yolo start ... 与 yolo init OK 两行（gpu init OK / firmware: CPU+GPU / boot ok 都不打）；每轮只打车牌字符串本身，不再有 #序号 / n / grab / infer 这些字段；没读出串的轮次一个字都不打；串口不处理按键（按键代码已从本交付树删除，收到的字节直接读掉丢弃），所以按任何键都不会有任何反应。要排查先把该宏改成 0 重新编译。另：camera init ERROR 同样被拦，相机起不来时的表现就是打完 yolo init OK 之后再没动静。

4.4 串口输入（本交付版无按键）

本交付版不接收任何串口命令。上电后固件直接进入定期扫描（见 4.3）：一轮结束歇 AUTO_SCAN_PERIOD_US（1 秒，定义见 3.5）再抓一帧跑一轮，循环往复；等待期间到达的串口字节只读掉丢弃（src/main.c 里的 (void)USART_ReceiveData(USARTx);），既不触发推理，也不改变任何开关。

要看到每轮更细的输出，改宏不改代码：把 inc/Message.h 的 DISPLAY_PLATE_ONLY 改成 0 重新编译，每轮就会打 #序号 / plate / n / grab / infer 那一行（格式见 4.3）。本交付版除这个编译宏之外，没有其它运行时开关。

4.5 典型使用流程

① 换模型上板（全部在【工程根目录】下执行）：

```
1) python tflite/export_onnx.py --kind detector --data training/data/voc.data \
       --weights training/weights/voc-299-epoch-0.270855ap-model.pth \
       --output tflite/onnx/plate.onnx
       # .pth -> onnx（export_onnx=True：输入 x、两个 85 通道输出，见 2.3）
       # 旧路线 pytorch2onnx_simplify.py + onnxsim 只作对照
2) python tflite/onnx2tf_int8.py --onnx tflite/onnx/plate.onnx \
       --calib training/data/val --size 320x320 \
       --out-dir tflite/onnx2tf_int8_plate
       # onnx -> int8 tflite（②③ 换各自的 onnx / calib 再跑一遍，见 2.4）
3) （--out-dir 已直接产出 plate-int8.tflite，本步不再需要）
4) python tflite/export_c_array.py --tflite tflite/onnx2tf_int8_plate/plate-int8.tflite \
        --array g_model_data --section data_area --out mcu_code/Printf2/inc/model_data.h
5) Keil 中构建：mcu_code/Printf2/MDK-ARM/Printf.uvprojx，确认 0 Error（V1.0.0 镜像另有 8 条告警，
```

```
全部来自 TFLite-Micro 厂商包）
6) 用 uVision 把 Printf.bin 下载到板子
```

可选的对账：1) 之后进 training/ 目录执行 python test_onnx_opt.py，直接看这份 ONNX 解出来的框（同一张 training/img/21.jpg 上应解出整牌框 + 7 个字符框，共 8 个目标），与固件侧同一张图的框对照。

② 上电验证：

```
- 不按任何键：应看到 firmware: CPU(+GPU) / boot ok（V1.0.0 交付版这两行也不打，只有 yolo start ... / yolo init OK），随后每约 2.7~3.5 秒一行车牌
- 交付版没有按键：按任何键都不会有反应（见 4.4）
- 要看每轮耗时与字符数，把 inc/Message.h 的 DISPLAY_PLATE_ONLY 改成 0 重新编译
```

③ 调试识别质量：

```
- 交付版没有运行时按键，串口输入不触发任何行为（见 4.4）
- 可用手段只有改宏：把 inc/Message.h 的 DISPLAY_PLATE_ONLY 改成 0 重新编译，拿到每轮 #n cas plate: … n … grab … infer … 的明细行
- 框 dump、几何分组、非卷积算子计时这些诊断入口不在本交付包内；需要时在源码里自行加打印
```

5. 常见问题

下表列出转换、编译、上板过程中最常遇到的几类现象与处理方式。

现象	处理
onnx2tf 报 ValueError: Depth of input (40) is not a multiple of input depth of filter (24)（node_Conv_1197）	ONNX 用的不是 fused 那一版。请用工程自带的 tflite/export_onnx.py（V1.0.0 版）重新导出；旧路线是 pytorch2onnx/pytorch2onnx.py 或 pytorch2onnx_simplify.py（它们显式传了 export_onnx=True，见 2.3），导出后按 2.3.3 的四项指纹核对
test_onnx.py / test_onnx_opt.py 提示“这个 ONNX 不是 fused 版本”	脚本检到输入名不是 x、或输出不是两个 85 通道张量。按 2.3 重新导出后再验证
转换 / 验证脚本报 ImportError: No module named 'torchvision'（或 'torchsummary'）	requirements.txt 里已声明 torchvision（0.28.0，配 torch 2.14）与 torchsummary 1.5.1。utils/utils.py 里只有 non_max_suppression 用得到 torchvision，已改成函数内延迟导入，所以只跑导出/量化（pytorch2onnx/pytorch2onnx.py、pytorch2onnx/pytorch2onnx_simplify.py）不会碰到它；走 NMS 的 test.py、test_onnx.py 需要它。torchsummary 由 training/model/backbone/shufflenetv2.py 顶层导入，凡是构造 Detector 的脚本都要它。按 2.2.2 建环境并 pip install -r requirements.txt 即可，单装则是 pip install torchvision==0.28.0 torchsummary==1.5.1
检测结果全错 / 一个框都没有	先核对输出布局：取 cat 之后的平面 blob，通道数等于 15+CLS（本项目 85），w/h 等于 320/stride
控制台 / 串口中文乱码	串口终端需按 GBK 解码；PC 端程序需 /utf-8 编译并 SetConsoleOutputCP(CP_UTF8)
在终端里选中文字准备复制，担心触发板子动作	本交付版不处理任何串口输入（按键代码已删除），选中文字不会有副作用；复制日志请用终端菜单里的“保存到文件”或鼠标拖选
编译报 g_model_data 重复定义	工程里除了 inc/model_data.h 又加入了另一份同名数组定义，把它移出工程即可（车牌模型只用 inc/model_data.h）
AllocateTensors 失败	tensor arena 不足。检查 TENSOR_ARENA_SIZE 与 M7_TCM.sct 中对应段的长度
访问 0xC0000000 区域触发 HardFault	该区域需按 TFLM 的要求配置 MPU 属性，以支持非对齐访问
运行转换脚本报 ImportError: No module named 'utils'	跑的通常不是工程里 pytorch2onnx/、tflite/ 下的这一份脚本（例如根目录下还留着旧副本）。工程自带的脚本按自身所在位置把 training/ 加进 sys.path，不依赖当前工作目录

版本历史

版本	日期	备注
V1.0.0	2026-10-22	初版发布（板端三网级联）